



Freightcarcheck

High-Level Architecture & Design

Proof of Concept | v1.0 | 2026

Prepared for Train Safety & Emergency Response Teams

1. Executive Summary

FreightcarcheckK is a web-based manifest viewer built for freight train crews, conductors, and first responders. It transforms dense, text-heavy train manifests into an at-a-glance visual dashboard — making critical hazmat, weight, and cargo data accessible in seconds rather than minutes.

A 150-car freight train can exceed 18,000 tons and stretch over two miles. In an emergency — derailment, stuck brakes, track fire — crews need instant answers: Where is the hazmat? What class? How heavy? FreightcarcheckK answers all three before the phone rings.

Audience	Primary Need	Key Benefit
Train Conductors	Pre-departure audit	Verify hazmat & weight in one view
First Responders	Hazmat location during incident	Rapid evacuation zone planning
Safety Teams	Regulatory compliance	DOT / FRA audit readiness

2. System Overview

The system follows a three-layer architecture: a data layer that holds train manifest records, a business logic layer that computes safety metrics, and a presentation layer that surfaces three purpose-built views to the user.

Layer	Responsibility
Presentation	Interactive web UI — Dashboard, Manifest Table, Train Layout tabs
Business Logic	Tonnage calculation, braking ratios, weight distribution, hazmat classification, search and filtering
Data	Train manifest records — car IDs, cargo descriptions, hazmat class, UN codes, weight, route data

Technology Stack

Component	Technology	Purpose
UI Framework	Streamlit (Python)	Browser-based interactive app

Charts & Diagrams	Plotly	Interactive charts, train layout rendering
Data Processing	Pandas / NumPy	Manifest parsing, metric computation
Data Storage	JSON	Train manifest format

3. User Roles & Interactions

Three distinct user personas interact with the application. Each arrives with a different intent and follows a different path through the interface.



Train Conductor

Goal: Conduct a pre-departure safety audit before the train leaves the yard.

- Opens the app and selects the assigned train from the sidebar dropdown
- Reviews the Dashboard KPIs — total cars, tonnage, hazmat count, braking ratio
- Checks the Manifest Table for correct UN codes and hazmat class assignments
- Scans the Train Layout diagram to confirm hazmat cars are not clustered at the front
- Uses the car type filter to confirm tank car positions relative to grades on the route



First Responder

Goal: Rapidly identify hazmat cars and establish a safe evacuation perimeter during a derailment.

- Accesses the app on-scene and selects the involved train
- Immediately activates the Hazmat-only toggle in the Manifest Table
- Reads DOT class, UN code, and car position for each hazmat car
- Switches to the Train Layout to visually locate red (hazmat) cars along the consist
- Uses hover tooltips to read exact tonnage per car for rescue planning



Safety Team

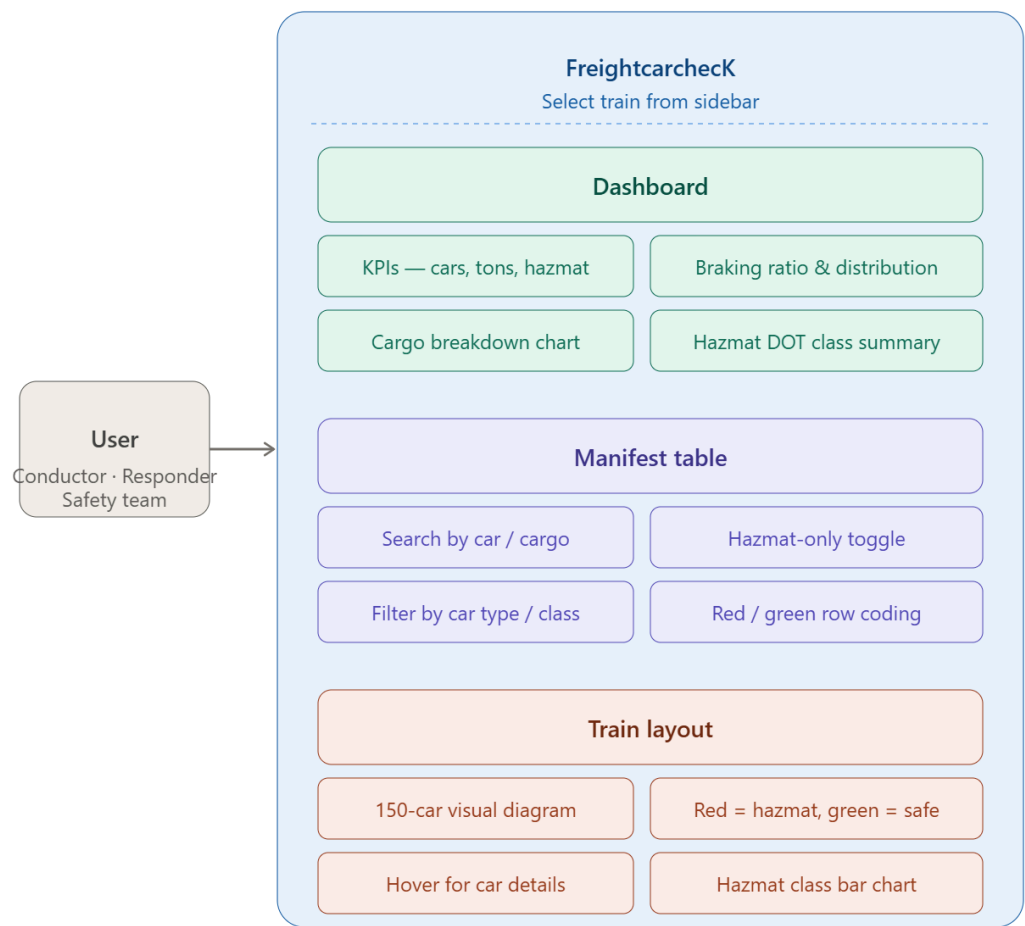
Goal: Validate regulatory compliance and optimise train handling for steep grade routes.

- Loads train manifest and reviews the braking ratio KPI on the Dashboard
- Checks front / middle / rear weight distribution percentages
- Filters the Manifest Table by hazmat class to validate DOT class assignments

- Reviews hazmat tonnage breakdown to flag disproportionate dangerous material loads
- Uses the visual layout to confirm heavy cars are distributed correctly for grade safety

4. Core Feature Blocks

The application is organised into three primary views. Each view is purpose-built for a different phase of the user workflow.



4.1 Dashboard

The Dashboard is the first screen users see after selecting a train. It provides a complete health summary of the consist in a single view.

Building Block	What It Shows	Who Uses It
----------------	---------------	-------------

KPI Cards	Total cars, total tonnage, hazmat car count, braking ratio	All roles — first glance summary
Cargo Breakdown Pie	Hazmat vs safe cargo tonnage split	Safety teams, first responders
Weight Distribution	Front / middle / rear weight percentages	Conductors, safety teams
Car Type Chart	Count of flat, tank, gondola, hopper, box cars	Conductors
Hazmat Summary Table	DOT Class 1-9 breakdown with tonnage	Safety teams, first responders

4.2 Manifest Table

The Manifest Table is the searchable and filterable record of all 150 cars. It is the operational reference view — where users look up specific cars or narrow down to a subset of the consist.

Building Block	What It Does	Who Uses It
Full-Text Search	Search by car ID, cargo description, origin, or destination	All roles
Hazmat-Only Toggle	Instantly filters to show only hazardous material cars	First responders (primary use)
Car Type Filter	Multiselect: flat, tank, gondola, hopper, box cars	Conductors, safety teams
Hazmat Class Filter	Narrows results to specific DOT Class (1 through 9)	Safety teams, first responders
Colour-Coded Rows	Red rows = hazmat cars; green rows = safe cargo	All roles — rapid visual scan

4.3 Train Layout

The Train Layout renders the full consist as an interactive horizontal diagram. It is the spatial view — showing where each car sits in the train, colour-coded by hazmat status.

Building Block	What It Shows	Who Uses It
150-Car Horizontal Diagram	All cars rendered as colour-coded bars (red = hazmat, green = safe)	All roles

Hover Tooltip	Car ID, position number, weight, cargo, hazmat class on hover	First responders (scene use)
Hazmat Distribution Chart	Bar chart of hazmat car count per DOT class	Safety teams, first responders

5. Key Safety Metrics

The application surfaces four critical safety metrics that are not visible in a raw manifest document. Each metric serves a specific operational decision.

Metric	Definition	Operational Decision
Total Tonnage	Sum of all car and cargo weights across the consist	Fuel planning, locomotive assignment, track weight limits
Hazmat Tonnage	Total weight of hazardous cargo only (not car weight)	Emergency response planning, evacuation radius, spill volume estimation
Braking Ratio	Hazmat tonnage divided by total tonnage	Determines required braking force on steep grades; prevents train separation
Weight Distribution	Front / middle / rear tonnage as percentage of total	Prevents excessive coupling stress and locomotive overload on grades

6. Data Model

Each train manifest is a structured record containing a header and a list of car entries. The key fields that drive all calculations and filters are listed below.

Train Record

Field	Type	Description
Train ID	String	Unique identifier e.g. TRAIN-28074
Manifest Date	Timestamp	Date and time the manifest was generated
Total Cars	Integer	Always 150 for a full consist
Total Weight	Decimal	Pre-computed sum of all car and cargo weights (tons)

Car Entry

Field	Type	Description
Car ID	String	Unique car identifier e.g. TANK-00042
Car Type	Enum	flat · tank · gondola · hopper · box
Position	Integer	Ordinal position in the consist (1–150)
Capacity (tons)	Decimal	Maximum rated load for the car type
Cargo Weight	Decimal	Actual cargo weight in tons
Is Hazmat	Boolean	True if car carries hazardous material
Hazmat Class	Integer	DOT Class 1–9 (null if not hazmat)
UN Code	String	UN number e.g. UN1072 (null if not hazmat)
Origin	String	Departure city / state
Destination	String	Destination city / state

6. Implementation Steps

Step 1 : In VS Code - Chat (Plan mode with Claude Haiku 4.5) Used Prompt Below “Freightcarcheck

The Heavy Haul Manifest Viewer
*Problem Statement*A 150-car freight train can weigh over 18,000 tons and stretch over two miles. Train crews (conductors and engineers) receive massive, text-heavy paper documents ("consists" or manifests) detailing cargo. In an emergency—like a derailment or a stuck brake—crews struggle to quickly locate hazardous materials (hazmat), calculate total tonnage, or determine braking weight ratios.*Why Need to Solve It?*Seconds matter during a derailment. First responders need immediate, accurate data on which cars contain hazardous chemicals to establish safe evacuation perimeters.*ROI (Return on Investment)*Disaster Mitigation: Prevents chemical spills from escalating by providing emergency crews with instant hazmat locations.*Regulatory Compliance:* Avoids massive federal fines for inaccurate or slow cargo reporting during safety audits.*Train Handling Optimization:* Accurate weight distribution data helps engineers prevent train break-in-twos on steep mountain grades.*Who Benefits & Users?**Train Conductors:* Can instantly audit their train’s composition and locate dangerous cars.*First Responders*

*(Fire/Hazmat Teams): Get a clear digital picture of the train layout before entering a crash site.
I'm thinking about an intuitive UI with static data for this poc app. UV app for easy proof & validation. ”*

Step 2 : As part of HITL (Human in The Loop) - answer the preferred choices for the questions

Analyzing your answers...

Q: What's your preferred tech stack for the UI?

A: **streamlit - UV python app**

Q: When you said 'UV app', did you mean a web app, desktop app, or something else?

A: **Its a python app with streamlit UI**

Q: What are the top 3 MVP features you'd prioritize?

A: **Display train manifest with car details, Highlight/filter hazmat (hazardous materials) cars, Show total tonnage & weight calculations, Search/find specific cars or cargo, Visual train layout diagram**

Q: How would you like to provide static manifest data for the POC?

A: **JSON file embedded in the app**

Q: Where/how should this run for validation?

A: **Local development server (<http://localhost>)**

Analyzed app requirements and planned data model questions

Perfect! Now I understand: **Streamlit Python app** with visual capabilities. One more clarification on the data model:

Analyzing your answers...

Q: Do you have an example or specific format for the train manifest? What details should each car record contain?

A: **I don't have. auto generate as needed for this app**

Q: How detailed should hazmat tracking be?

A: **Include hazmat classification (Class 1-9, UN codes)**

Created comprehensive application plan and saved it

Step 3 : Detailed Implementation plan and Approvals.

Plan: Freightcarcheck — Heavy Haul Manifest Viewer**TL;DR**

Build a Streamlit Python app that serves train crews and first responders an intuitive interface for viewing freight manifests. The POC features:

- Auto-generated realistic 150-car train data with hazmat classification (DOT Classes 1-9 + UN codes)
- Tabbed interface: Dashboard (KPIs), Manifest (searchable/filterable table), Train Layout (visual diagram)
- Tonnage calculations, hazmat filtering, and braking weight ratios
- Local Streamlit server for rapid validation

Steps**Phase 1: Project Foundation & Data Model**

1. Initialize project structure *(parallel with step 2)*
 - Folders: `app.py`, `utils/`, `data/`, `.streamlit/config.toml`
 - `requirements.txt`: streamlit, pandas, plotly, numpy
 - `.gitignore`
2. Define data model *(parallel with step 1)*
 - `models.py` with dataclasses: `Train`, `Car`, `Cargo`, `HazmatInfo`
 - Car: ID, type (flat/tank/gondola), capacity_tons, actual_weight, origin, destination
 - Cargo: description, weight, is_hazmat flag, hazmat_class (1-9), UN code
3. Auto-generate realistic sample data
 - `data_generator.py`: Create 150-car trains with realistic weight distribution
 - Tank cars ~140 tons, flat cars ~30 tons, etc.
 - ~15-20% hazmat (Classes 3, 4, 5, 8, 9)
 - Output: `data/sample_trains.json` (2-3 sample manifests)

Phase 2: Core Utilities *(depends on Phase 1)*

4. Build calculations — `utils/calculations.py`
 - Total tonnage, hazmat tonnage, weight distribution (front/middle/rear %), braking ratio
5. Build filters & search — `utils/filters.py`
 - Filter by hazmat class, car type; search cars by ID/cargo/origin/destination

Relevant Files

- `app.py` — Main entry point, tabs layout
- `models.py` — Data classes (Train, Car, Cargo, HazmatInfo)
- `data_generator.py` — `generate_realistic_train(num_cars=150)` function
- `data/sample_trains.json` — Pre-generated 150-car manifests
- `utils/calculations.py` — Tonnage, braking ratio, weight distribution
- `utils/filters.py` — Hazmat/car-type filters, search
- `utils/visualizations.py` — Plotly train layout diagram and charts
- `requirements.txt`, `streamlit/config.toml`

Verification Checklist

1. Data generation: `sample_trains.json` contains 150 cars with realistic weights & hazmat mix
2. App launch: `streamlit run app.py` → opens on localhost:8501 without errors
3. Dashboard: KPI totals match manual calculations
4. Search: Filter for specific car ID → shows only that car
5. Hazmat toggle: "Hazmat Only" shows only hazmat cars; count matches dashboard
6. Train Layout: All 150 cars render; hazmat=red, safe=green
7. Color consistency: Red/green colors match between table and layout visualization

Key Decisions

- Architecture: Single-page multi-tab (not multi-page routes) for POC simplicity
- Data: JSON embedded in app (not CSV upload) for guaranteed consistency
- Visualization: Plotly for interactive train layout with hover details
- Hazmat Standard: DOT Class 1-9 + UN codes (US freight standard)
- Sample Data: Auto-generated (not hardcoded) to test at scale

Step 4 : Approval for Implementation (*Agent Mode - Claude Haiku 4.5*)

Start implementation

Optimized tool selection

Created project structure and implemented initial files

I'll start implementing the FreightcarcheckK application. Let me begin with the project structure and core files.

Created 5 todos and added files

Now creating core files:

Created 5 files

Created 5 files

Step 5 : Implementation completed and run application steps

Project Structure

```
freightcarcheck/
├─ app.py                # Main Streamlit app
├─ models.py             # Data models
├─ data_generator.py     # Sample data (150-car trains)
├─ validate.py           # Component tests (ALL PASS)
├─ utils/
│  ├─ calculations.py    # Tonnage, ratios, distributions
│  ├─ filters.py         # Search & filtering logic
│  └─ visualizations.py  # Plotly charts
├─ data_loader.py        # JSON to DataFrame
├─ data/
│  └─ sample_trains.json # 3 realistic manifests
├─ .streamlit/config.toml # Streamlit settings
├─ requirements.txt      # All dependencies
├─ run.bat               # Windows quick launch
├─ QUICKSTART.md         # Setup guide
└─ README.md             # Full documentation
```

Validation Results

Imports..... PASS

Data Generation..... PASS (150 cars, 27 hazmat)

Data Loading..... PASS (3 trains, 450 cars, calculations correct)

Filters..... PASS (search, hazmat, car types working)

Quick Start

```
# Windows (easiest)
run.bat

# Or manual
python -m pip install --user -r requirements.txt
python data_generator.py
```

Step 6: **Fix Error** raised by application when initiating application on localhost , Troubleshoot error using VScode chat (Agent Mode - Claude Haiku 4.5)

```
module 'streamlit' has no attribute 'subtitle'
Traceback:
File "C:\Users\Sunny\Documents\Trevor\workspace\freightcarcheck\app.py", line 41, in <module>
  st.subtitle("Heavy Haul Manifest Viewer - Emergency Response Support")
  ^^^^^^^^^^^
```

This is error on localhost browser

Updated app.py +1 -1

Now let me check the browser to see if the error is resolved:

Excellent!  The error is fixed! The app is now running perfectly. Here's what I can see:

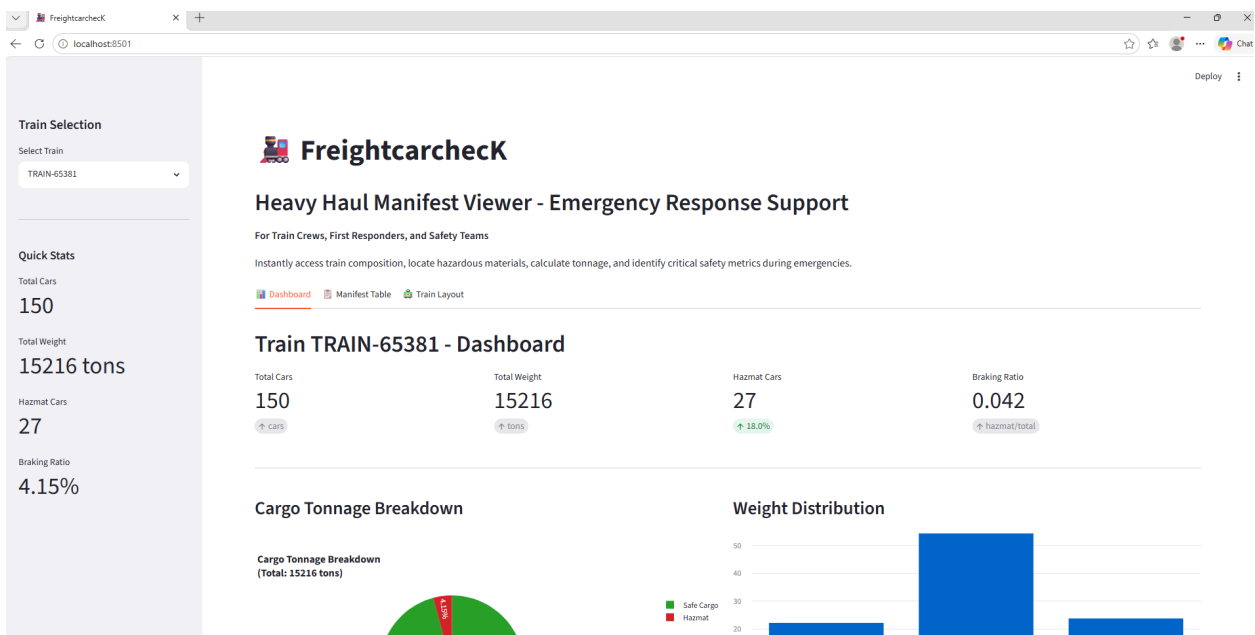
-  Title: "  Freightcarcheck" displays correctly
-  Subtitle: Now renders as a proper heading using markdown instead of `st.subtitle()`
-  Sidebar: Shows Train Selection (TRAIN-65381) and Quick Stats (150 cars, 15216 tons, 27 hazmat cars)
-  Three Tabs: Dashboard (active), Manifest Table, Train Layout

Step 7 : Open terminal in VS code and launch app.

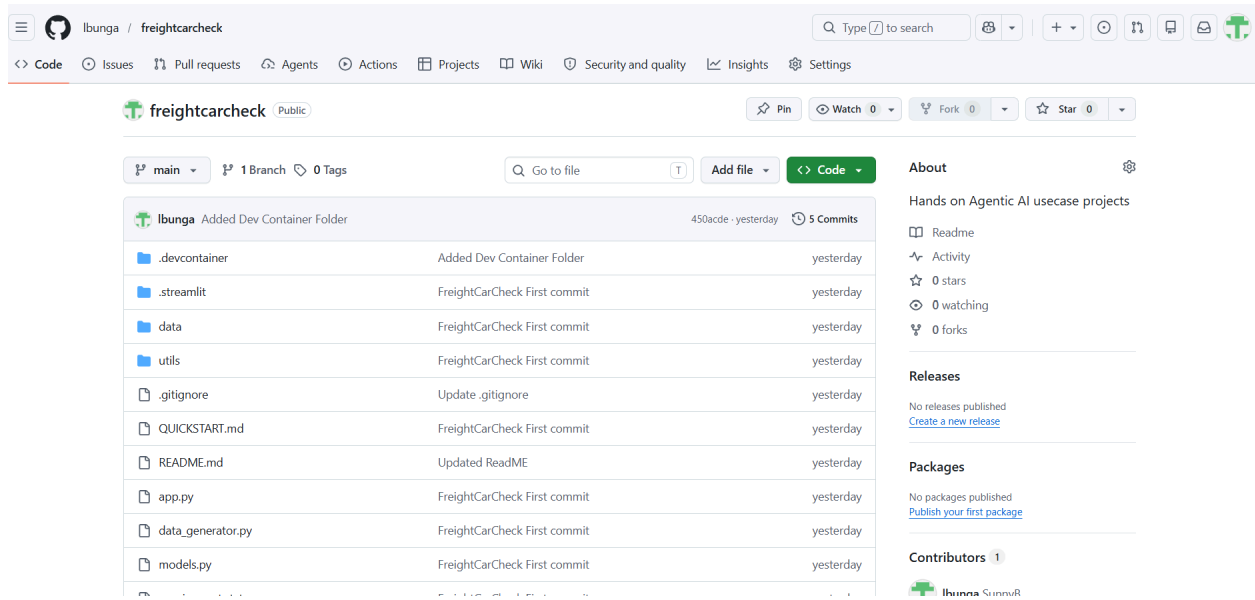
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
(.venv) PS C:\Users\Sunny\Documents\Trevor\workspace\freightcarcheck> streamlit run app.py
```

Step 8 : Validate app on Local URL: <http://localhost:8501>



Step 9 : Upload Application to github using Github Desktop Client



Step 10 : Using <https://streamlit.io/> and Github integration deployed Git Code

This is App URL : <https://freightcarcheck-tzfgemq4ghovntjm9ykges.streamlit.app/>